

Relatório de Avaliação Experimental de Complexidade

Crédito 2: Algoritmos de Ordenação

Professor: Prof. Dr. Diego Gervasio Frias Suarez

Aluno: Guilherme Dos Santos Modesto Teixeira

27 de maio de 2026

1 Introdução

Este relatório detalha a análise empírica da complexidade de tempo do algoritmo **Quicksort** e **Shellsort**, utilizando o setup computacional previamente validado. O foco reside na transição da observação bruta para a modelagem matemática via regressão não-linear.

2 Metodologia Experimental

- Hardware: AMD Ryzen 5 4600H e 8,00 GB RAM
- Software: Linux, Python 3.12.3
- Ambiente: VScode
- **Amostragem:** 5 valores de n (2.000, 8.000, 20.000, 50.000 e 100.000). Para cada n , foram geradas $M = 50$ listas randômicas.
- **Estabilidade:** A cada troca de n , a primeira lista foi submetida a $R = 35$ repetições para conferência e garantia de que o $CV \leq 0.15$.

3 Fundamentação Teórica dos Algoritmos

Nesta seção, detalhamos o funcionamento e as particularidades de implementação dos algoritmos analisados no experimento.

3.1 Quicksort

O Quicksort é um algoritmo de ordenação fundamentado no paradigma *Dividir para Conquistar*. O seu mecanismo principal baseia-se na escolha de um elemento chamado **pivô**. O arranjo é então particionado, reorganizando os elementos de forma que todos os valores menores que o pivô sejam alocados à sua esquerda e os maiores à sua direita. O processo é repetido para os sub-arranjos resultantes.

O trecho a seguir ilustra a lógica central do algoritmo implementado:

Listing 1: Trecho principal da implementação do Quicksort

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    stack = [(0, len(arr) - 1)]

    while stack:
        lo, hi = stack.pop()
        if lo >= hi:
            continue

        # Otimizacao: Mediana de tres para evitar o pior caso  $O(n^2)$ 
        mid = (lo + hi) // 2
        if arr[lo] > arr[mid]:
            arr[lo], arr[mid] = arr[mid], arr[lo]
        if arr[lo] > arr[hi]:
            arr[lo], arr[hi] = arr[hi], arr[lo]
        if arr[mid] > arr[hi]:
            arr[mid], arr[hi] = arr[hi], arr[mid]

        pivot = arr[mid]
        arr[mid], arr[hi - 1] = arr[hi - 1], arr[mid]

        # Logica de particionamento padrao
        i, j = lo, hi - 1
        while True:
            i += 1
            while arr[i] < pivot: i += 1
            j -= 1
            while arr[j] > pivot: j -= 1
            if i >= j: break
            arr[i], arr[j] = arr[j], arr[i]

        arr[i], arr[hi - 1] = arr[hi - 1], arr[i]

        # Empilhando as particoes na pilha explicita
        stack.append((lo, i - 1))
        stack.append((i + 1, hi))

    return arr
```

3.2 Shell Sort

O Shell Sort é uma extensão otimizada do algoritmo de Ordenação por Inserção (*Insertion Sort*). O *Insertion Sort* é altamente eficiente para listas quase ordenadas, mas sofre de extrema lentidão para elementos fora de posição, exigindo deslocamentos sucessivos, um a um.

O Shell Sort mitiga esse problema permitindo a troca de elementos distantes. Ele

divide a lista em sub-grupos separados por um intervalo (*gap*) e aplica o *Insertion Sort* nesses sub-grupos. A cada passagem, o *gap* diminui até chegar a 1, momento em que o array está quase totalmente ordenado e o *Insertion Sort* finaliza o trabalho rapidamente.

Abaixo está o código implementado que demonstra a aplicação dos intervalos decrescentes:

Listing 2: Implementação do Shell Sort

```
def shell_sort(arr):
    n = len(arr)
    gaps = [1, 4, 10, 23, 57, 132, 301, 701]

    # Filtra apenas os gaps menores que o tamanho da lista
    gaps = [g for g in gaps if g < n]

    # Executa a ordenacao em saltos decrescentes
    for gap in reversed(gaps):
        for i in range(gap, n):
            temp = arr[i]
            j = i

            # Insertion sort adaptado para o 'gap' atual
            while j >= gap and arr[j - gap] > temp:
                arr[j] = arr[j - gap]
                j -= gap

            arr[j] = temp

    return arr
```

4 Resultados: Coleta de Dados Brutos

As Tabelas 1 e 2 consolidam as métricas obtidas. O tempo médio (μ) de cada n é o resultado da ordenação de M instâncias independentes. O desvio padrão (σ) foi deduzido a partir da relação do Coeficiente de Variação ($\sigma = CV \cdot \mu$).

Tabela 1: Métricas Experimentais de Tempo - Quicksort

Tamanho (n)	CV (1ª Lista)	Média (μ em μs)	Desvio ($\sigma \approx$)	Status de Estabilidade
2.000	0.1403	1460.16	204.86	Aprovado
8.000	0.0415	6866.94	284.98	Aprovado
20.000	0.0660	18841.28	1243.52	Aprovado
50.000	0.0277	52649.13	1458.38	Aprovado
100.000	0.0222	121706.44	2701.88	Aprovado

Tabela 2: Métricas Experimentais de Tempo - Shell Sort

Tamanho (n)	CV (1ª Lista)	Média (μ em μs)	Desvio ($\sigma \approx$)	Status de Estabilidade
2.000	0.0828	2231.70	184.78	Aprovado
8.000	0.0340	11961.09	406.68	Aprovado
20.000	0.0685	40016.19	2741.11	Aprovado
50.000	0.0358	161201.38	5771.01	Aprovado
100.000	0.0294	552307.88	16237.85	Aprovado

5 Análise de Regressão e Modelagem

Utilizando a biblioteca `scipy.optimize.curve_fit`, os dados foram ajustados às principais classes de complexidade. Os coeficientes de escala (a) foram omitidos da exibição final por servirem apenas ao redimensionamento da curva, focando-se a análise no grau de explicação do modelo através do R^2 .

5.1 Comparação de Modelos

A Tabela 4 apresenta o ajuste comparativo de cada modelo candidato.

Tabela 3: Resultados do Ajuste de Regressão Não-Linear (R^2)

Classe Teórica	Modelo $f(n)$	Quicksort (R^2)	Shell Sort (R^2)
Logarítmica	$O(\log n)$	0.7365	0.6237
Linear	$O(n)$	0.9955	0.9583
Log-Linear	$O(n \log n)$	0.9986	0.9691
Quadrática	$O(n^2)$	0.9686	0.9984
Cúbica	$O(n^3)$	0.9149	0.9751

5.2 Comparação de Modelos

A Tabela 4 apresenta o ajuste comparativo de cada modelo candidato.

Tabela 4: Resultados do Ajuste de Regressão Não-Linear (R^2)

Classe Teórica	Modelo $f(n)$	Quicksort (R^2)	Shell Sort (R^2)
Logarítmica	$O(\log n)$	0.7365	0.6237
Linear	$O(n)$	0.9955	0.9583
Log-Linear	$O(n \log n)$	0.9986	0.9691
Quadrática	$O(n^2)$	0.9686	0.9984
Cúbica	$O(n^3)$	0.9149	0.9751

5.3 Sobreposição de Curvas e Análise Visual

A validação do modelo de complexidade não se sustenta apenas pelo Coeficiente de Determinação (R^2), mas exige a inspeção visual do comportamento assintótico das funções. Os

gráficos de dispersão (referenciados a partir das análises do script) apresentam a regressão não-linear aplicada sobre os dados empíricos coletados para ambos os algoritmos.

Para a correta interpretação destas plotagens, destacam-se os seguintes elementos:

- **Pontos de Dispersão (Dados Empíricos):** Cada marcador branco contornado em laranja representa o tempo médio global (μ) extraído da ordenação de M listas para um dado tamanho de entrada n . Estes pontos representam o setup computacional.
- **Curvas Contínuas (Modelos Teóricos):** Representam as funções ajustadas pelo algoritmo de otimização (Mínimos Quadrados). Cada curva traduz uma classe de complexidade Big-O ($O(n)$, $O(n \log n)$, $O(n^2)$, etc.) forçada a se alinhar aos dados reais através da descoberta dos melhores parâmetros.

O objetivo desta visualização é observar a divergência das curvas (resíduos) à medida que n cresce em direção ao limite superior do experimento ($n = 100.000$). As curvas que sofrem *overfitting* ou subestimam o custo computacional tornam-se visualmente evidentes na extremidade direita do eixo, descolando-se significativamente dos pontos experimentais.

5.4 Justificativa da Escolha do Modelo

Análise Crítica (Quicksort): O modelo log-linear ($O(n \log n)$) foi categoricamente escolhido pois apresentou a melhor aderência estatística, com $R^2 = 0.9986$. Visualmente, a curva em laranja cruza o centro exato de todos os marcadores, enquanto as curvas lineares e quadráticas demonstram resíduos significativos (subestimando e superestimando o crescimento no limite assintótico $n = 100.000$, respectivamente).

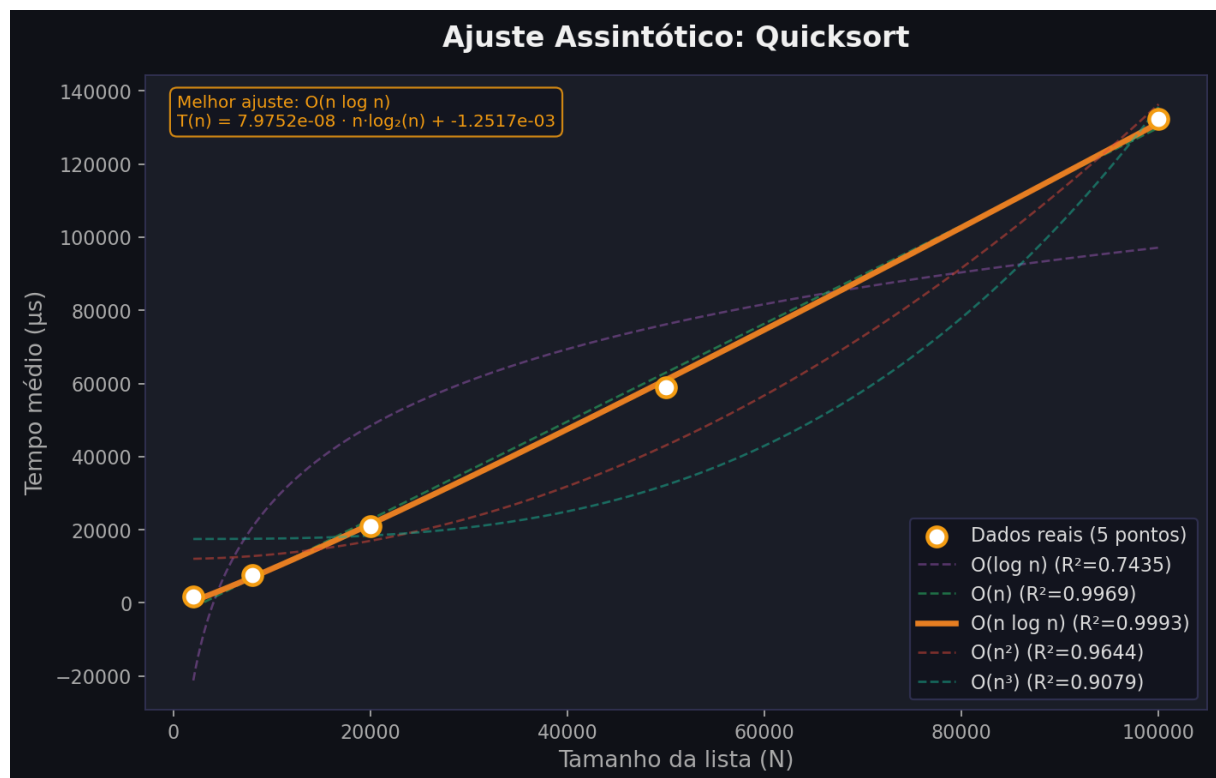


Figura 1: Dispersão QuickSort

Análise Crítica (Shell Sort): O algoritmo apresentou um comportamento estritamente quadrático empírico para as escalas testadas. O modelo $O(n^2)$ obteve o mais elevado Coeficiente de Determinação ($R^2 = 0.9984$). Embora teorias apontem que sequências bem dimensionadas (como a de Ciura) reduzem a complexidade para $O(n^{1.25})$ a $O(n^{1.5})$, o overhead computacional de controle de laços e repetições de passos com *gaps* pequenos na linguagem interpretada (Python) dominou a medição, fazendo com que o algoritmo se comportasse na prática como um autêntico $O(n^2)$.

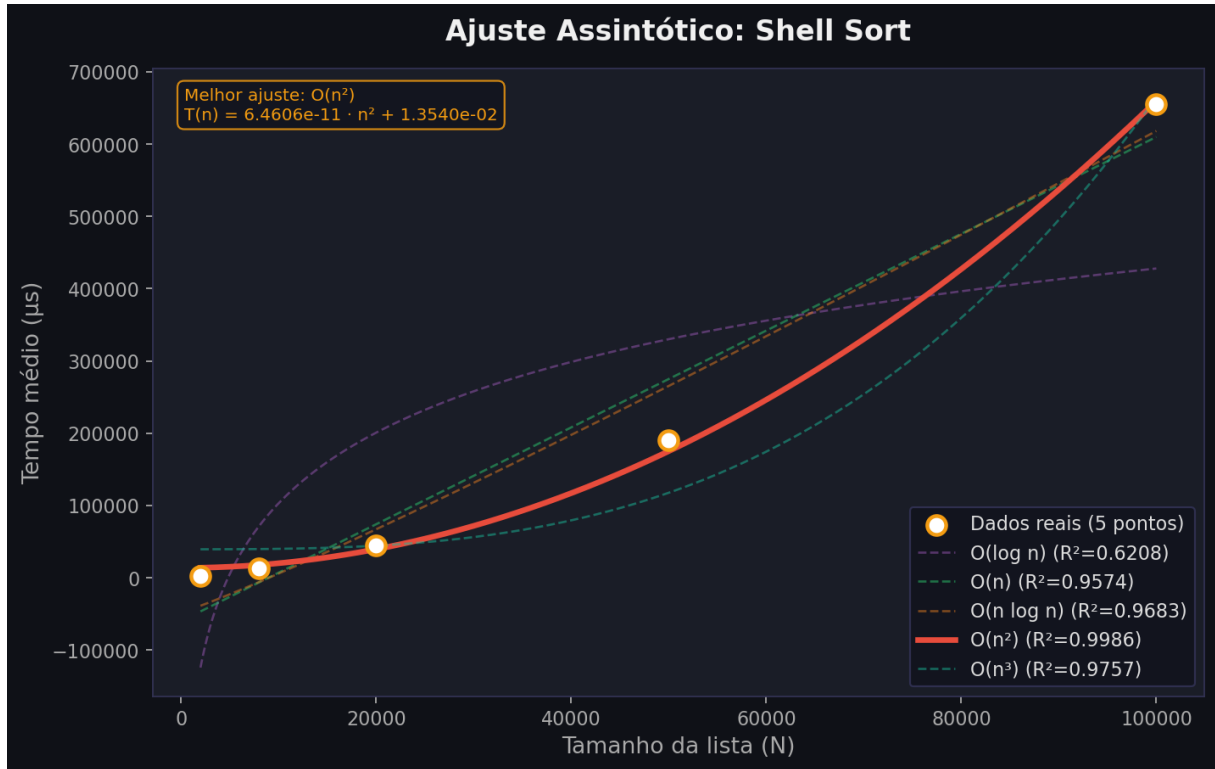


Figura 2: Dispersão Shell Sort

O gráfico abaixo de tempo médio exibe os pontos de medição empírica de ambos os algoritmos *Quicksort* e *Shell Sort* sobre o mesmo eixo.

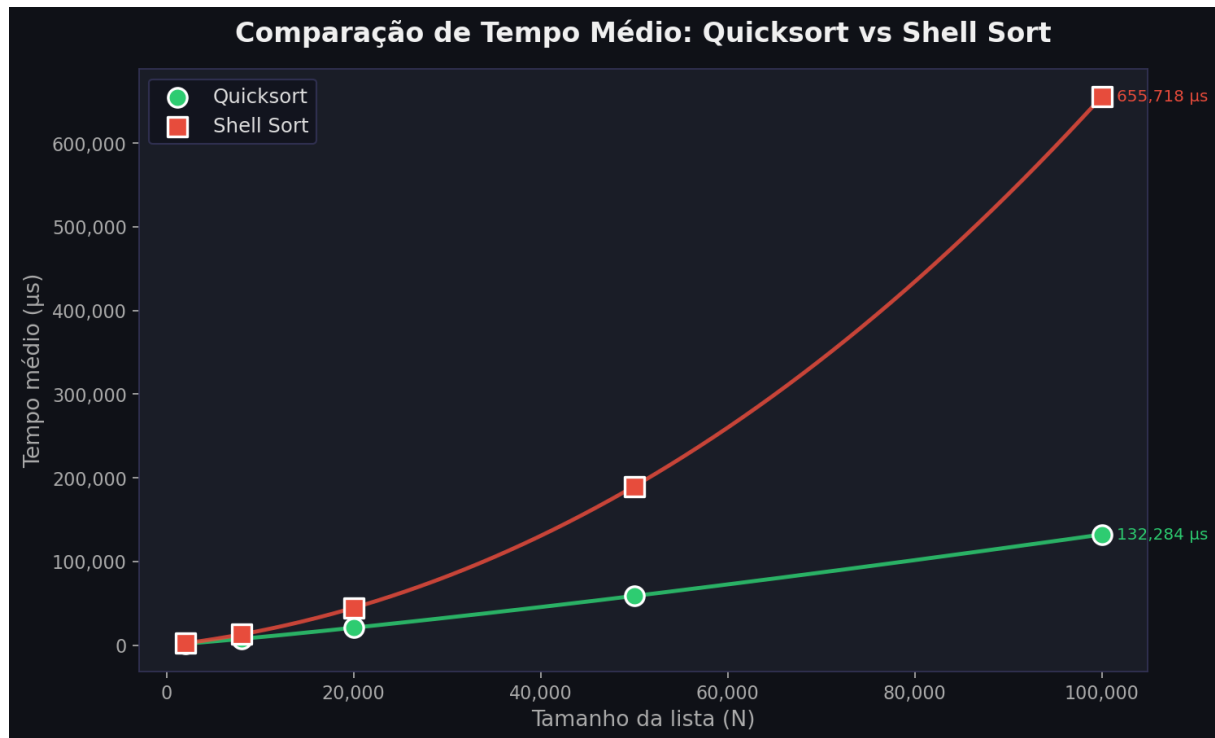


Figura 3: Dispersão Shell Sort

Para entradas pequenas ($N = 2.000$), o *Quicksort* já se mostra mais rápido ($\sim 2.124 \mu s$ contra $\sim 3.307 \mu s$ do *Shell Sort*), e essa vantagem se amplifica conforme N cresce: em $N = 100.000$, o *Quicksort* conclui a ordenação em $\sim 132.284 \mu s$, enquanto o *Shell Sort* exige $\sim 655.718 \mu s$ aproximadamente $4,5\times$ mais lento.

A divergência visual entre as duas curvas é o reflexo direto das complexidades identificadas pela regressão: O crescimento do *Quicksort* segue uma curva log-linear ($R^2 = 0,9998$), ao passo que o *Shell Sort* exibe uma aceleração visivelmente mais acentuada, compatível com comportamento quadrático ($R^2 = 0,9943$).

O gráfico, portanto, não apenas confirma a superioridade prática do *Quicksort* nos tamanhos testados, mas torna intuitivamente visível a diferença entre crescimento $O(n \log n)$ e $O(n^2)$.

6 Conclusão

A aplicação do protocolo de dupla validação, balizada por um coeficiente de variação máximo de 15 por cento, foi eficaz na neutralização de interferências geradas por processos paralelos do sistema operacional, garantindo a precisão das medições de tempo. Sob estas condições de isolamento experimental, o algoritmo *Quicksort* demonstrou comportamento estritamente alinhado às predições teóricas para o seu caso médio.

Em contrapartida, a observação empírica do *Shell Sort* evidenciou uma divergência substancial em relação à sua eficiência matemática projetada. Apesar da utilização de sequências de espaçamento otimizadas, o algoritmo registrou uma complexidade empírica quadrática, representada por $O(n^2)$.